

Desarrollo Fullstack Usando Arquitecturas de Microservicios

Tecnológico Nacional de México Campus Cd. Madero
Agosto 2026

Introducción a la Arquitectura de Microservicios

Arquitectura de microservicios

Los microservicios consisten en dividir una aplicación en pequeños servicios independientes que colaboran entre sí mediante APIs.

Cada servicio:

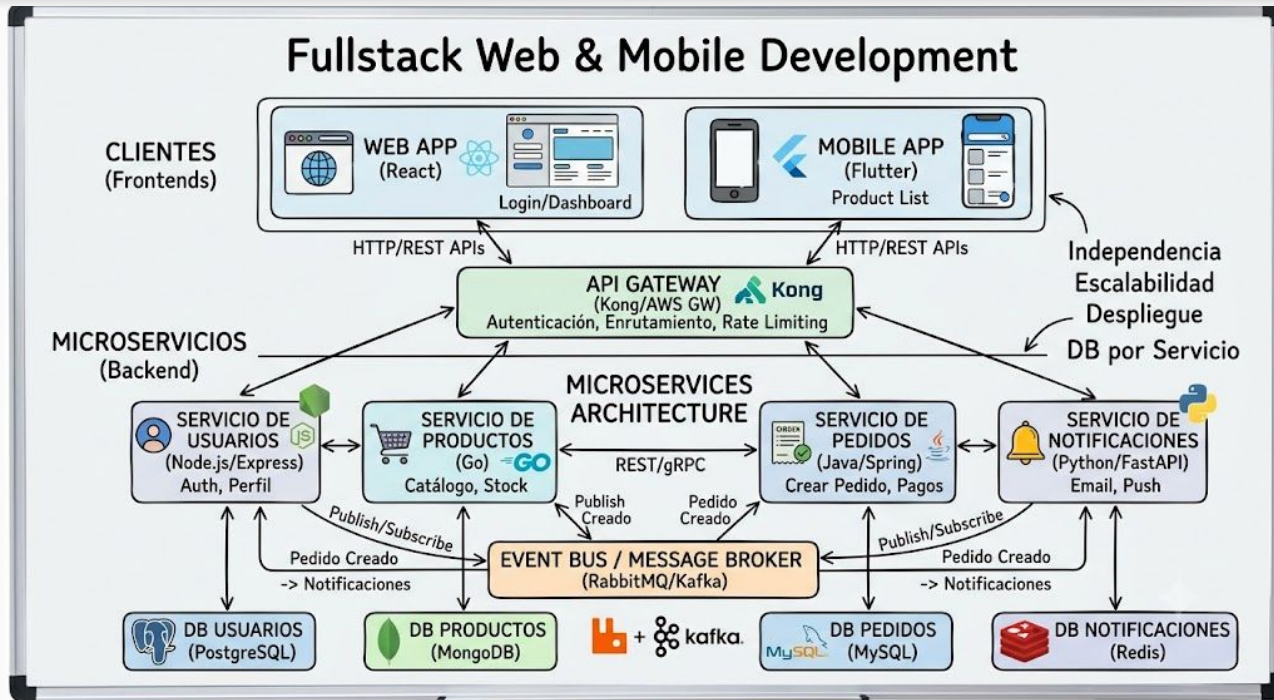
- Tiene su propia responsabilidad.
- Tiene su propia base de datos (idealmente).
- Puede desplegarse de manera independiente.
- Puede escalar de manera independiente.
- Está acoplado débilmente con el resto de la aplicación.
- Puede desarrollarse por equipos independientes.

El caso del comercio electrónico

Es un dominio complejo, el cual incluye:

- Gestión de inventario.
- Pedidos.
- Logística: distribución, envíos, devoluciones, entre otros.
- Cancelaciones, devoluciones y reembolsos.
- Facturación.
- Crédito y cobranza.
- Marketplace.

Arquitectura de microservicios



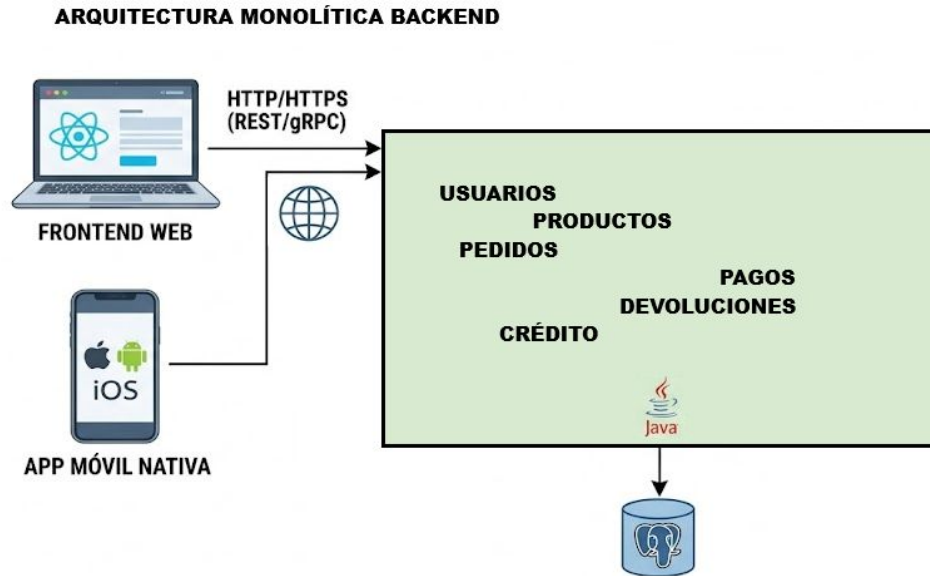
Arquitectura monolítica

Una aplicación monolítica concentra toda la lógica del negocio en un solo proyecto.

Sus principales desventajas:

- Crece demasiado.
- Difícil mantenimiento.
- Equipos grandes trabajan sobre el mismo código.
- Escalamiento limitado.
- Un error puede afectar toda la aplicación.

Arquitectura monolítica



REST API

Una REST API permite que diferentes aplicaciones intercambien información mediante HTTP utilizando recursos identificados por URLs.

- Es el acrónimo en inglés de Application Program Interface REpresentational State Transfer.
- Utilizan HTTP/HTTPS como protocolo de comunicación.
- La API se establece por medio de endpoints que son URLs asignadas a recursos específicos.
- La interacción entre cliente y servidor es por medio de los verbos HTTP: Post, Get, Put y Delete principalmente.
- Soporta como tipo de datos JSON, XML y otros formatos menos utilizados.

REST API y los Códigos HTTP

Un código HTTP es un número de tres dígitos que un servidor web envía como parte de la respuesta a una petición hecha por el navegador o cliente. Se dividen en cinco grupos según su primer dígito:

- **1xx (Informativos):** El servidor recibió la petición y sigue procesándola.
- **2xx (Éxito):** La solicitud se recibió, se entendió y se aceptó de forma correcta.
- **3xx (Redirección):** Se necesita hacer algo más para terminar la solicitud.
- **4xx (Error del cliente):** La petición tiene un error o no se tiene permiso para verla.
- **5xx (Error del servidor):** El servidor falló y no pudo cumplir con una petición

REST API y los Códigos HTTP

Códigos más comunes:

- 200 OK: Todo salió bien y la página cargó.
- 301 Movido permanentemente: La página cambió de dirección para siempre.
- 400 Solicitud incorrecta: El servidor no entiende la petición por un fallo de sintaxis.
- 401 No autorizado: Falta iniciar sesión o dar credenciales válidas.
- 403 Prohibido: No tienes permiso para ver ese contenido.
- 404 No encontrado: La página o archivo no existe.
- 500 Error interno del servidor: Hubo un fallo general en el servidor.

Frameworks backend para desarrollo web

Se ocupan de la lógica del servidor, el procesamiento de la base de datos, la autenticación de usuarios y la seguridad.

- Spring (Java, Kotlin)
- Django (Python)
- Express.js (Javascript)
- Laravel (PHP)
- Ruby on rails (Ruby)
- ASP.Net core (.NET)

Frameworks frontend para desarrollo web

Se usan para el desarrollo de la interfaz gráfica y de la experiencia del usuario.

- Java server faces (Java)
- React(Javascript)
- Angular (Javascript)
- Vue.js (Javascript)
- Blazor (.NET)
- Razor (.NET)

Frameworks fullstack para desarrollo web

Combinan o integran backend y frontend en un solo entorno.

- Spring con el uso de plantillas (Java, Kotlin)
- Java server faces (Java)
- Django (Python)
- Next.js (Javascript)
- Nuxt.js (Javascript)
- Meteor (Javascript)
- ASP.Net core (.NET)

Spring framework

Proporciona un modelo integral de programación y configuración para aplicaciones empresariales modernas basadas en Java, en cualquier plataforma de implementación.

Características principales:

- Tecnologías principales: dependency injection, events, resources, i18n, validation, data binding, type conversion, SpEL, AOP.
- Pruebas: mock objects, TestContext framework, Spring MVC Test, WebTestClient.
- Acceso a datos: transactions, DAO support, JDBC, ORM, Marshalling XML.
- Arquitecturas: Spring MVC and Spring WebFlux web frameworks.
- Integración: remoting, JMS, JCA, JMX, email, tasks, scheduling, cache and observability.
- Lenguajes de programación: Java, Kotlin, Groovy, dynamic languages.
- <https://spring.io/projects/spring-framework>

Spring boot

Facilita la creación de aplicaciones Spring autónomas y listas para producción, que puedes ejecutar sin complicaciones. La mayoría de las aplicaciones Spring Boot requieren una configuración mínima de Spring.

Características:

- Crea aplicaciones Spring independientes
- Integra servidores Tomcat, Jetty o Undertow, sin necesidad de desplegar archivos WAR
- Proporciona dependencias iniciales de una amplia variedad de bibliotecas de terceros
- Configura automáticamente Spring
- Proporciona funcionalidades listas para producción, como métricas, comprobaciones de estado y configuración externa
- No genera código y no requiere configuración XML
- <https://start.spring.io/>

Nuestro ambiente de desarrollo

- JDK 25: <https://www.oracle.com/java/technologies/javase/jdk25-archive-downloads.html>
- SQL Server Standard Developer:
<https://go.microsoft.com/fwlink/?linkid=2344626&clcid=0x40A&culture=es-es&country=es>
- SQL Server Management Studio: https://aka.ms/ssms/22/release/vs_SSMS.exe
- IDEs
 - ◆ IntelliJ: <https://www.jetbrains.com/idea/download/?section=windows>
<https://www.jetbrains.com/idea/download/?section=mac>
<https://www.jetbrains.com/idea/download/?section=linux>
 - ◆ Theia: <https://theia-ide.org/#theiaide>
 - ◆ Visual Studio Code: https://code.visualstudio.com/download?_exp_download=fb315fc982
 - ◆ Desde la página de Spring: <https://spring.io/tools>
- Google Chrome o FireFox

Nuestro ambiente de desarrollo

- Gradle: <https://www.oracle.com/java/technologies/javase/jdk25-archive-downloads.html>
- Maven: <https://go.microsoft.com/fwlink/?linkid=2344626&clcid=0x40A&culture=es-es&country=es>
- Github: <https://github.com/>
- Git: <https://git-scm.com/install/>
- Docker: <https://www.docker.com/>
- Postman: <https://www.postman.com/> o HTTPie: <https://httpie.io/>

Primer proyecto

Diseñar e implementar una aplicación web/móvil nativa basada en microservicios para un catálogo de productos con los siguientes atributos:

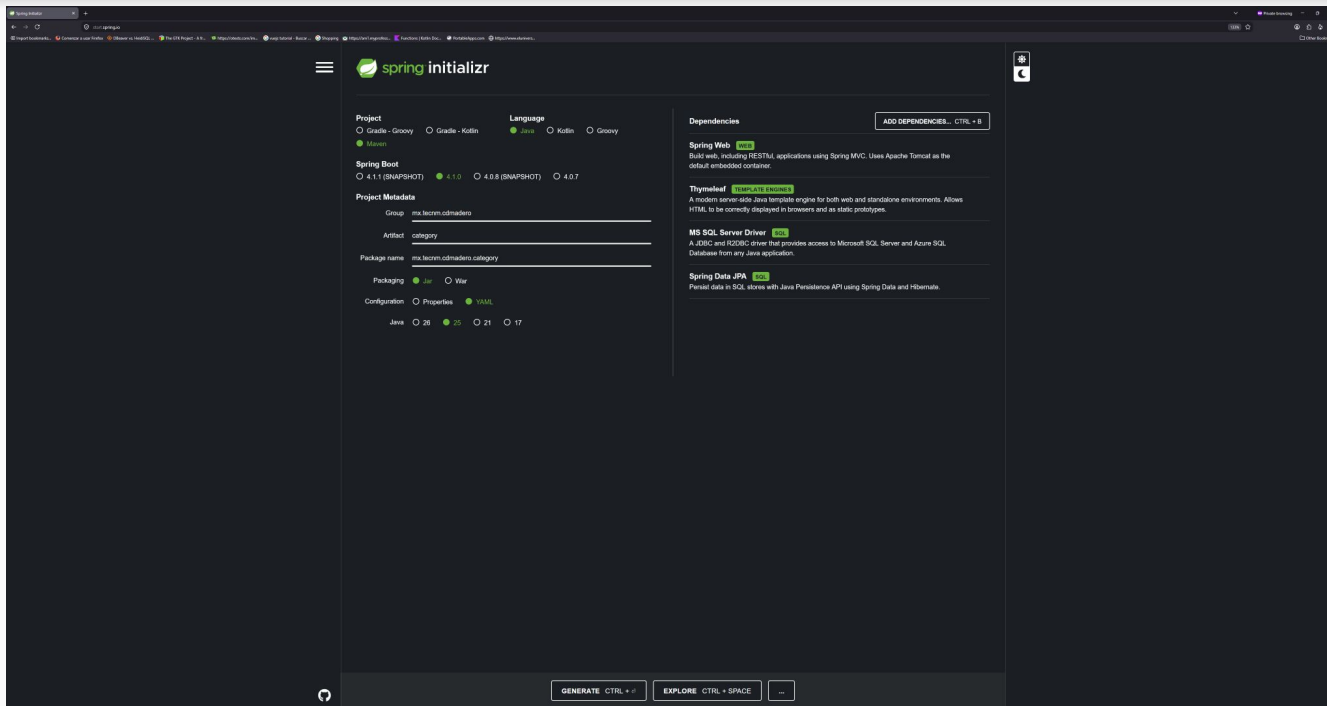
product_id	int	primary key, auto incrementable
product_name	nvarchar(40)	not null
quantity_per_unit	nvarchar(20)	not null
unit_price	money	not null, default 0
units_in_stock	int	not null, default 0
reorder_level	int	not null, default 0
category	Category	not null
discontinued	boolean	not null, default false

Primer proyecto

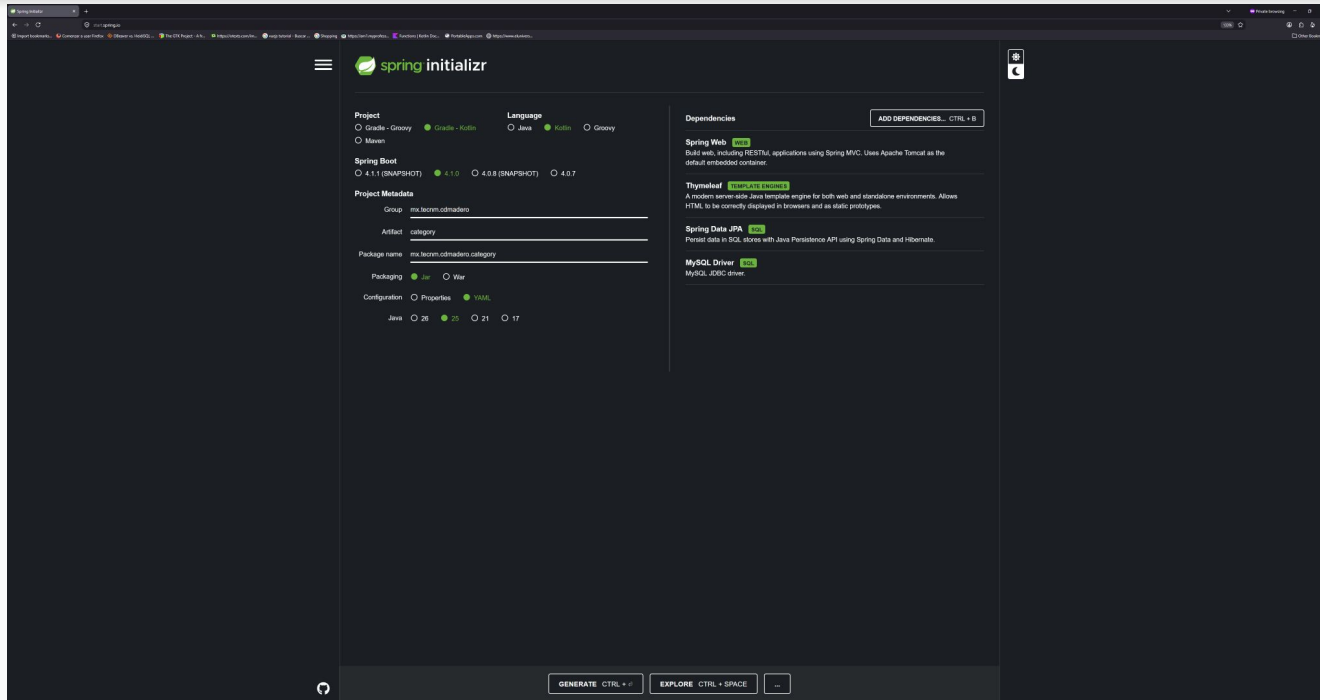
La categoría del producto cuenta con los siguientes atributos:

category_id	int	primary key
category_name	nvarchar(40)	not null
description	nvarchar(20)	not null
tax	money	not null
picture	image	not null

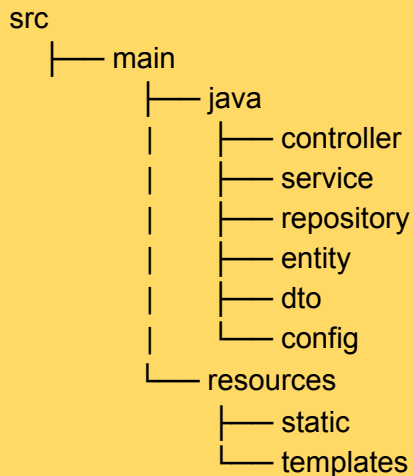
Fullstack de categorías de productos



Fullstack de producto con Spring boot



Estructura típica



En una aplicación web, el folder src organiza el código del backend en subfolders separando las tareas en capas, cada uno con una función única para mantener el orden:

→ Controladores y Servicios

- ◆ Controller (Controlador): Recibe las peticiones del usuario, valida los datos básicos y manda la respuesta final.
- ◆ Service (Servicio): Contiene la lógica del negocio, procesa la información y toma las reglas de decisión de la aplicación.

→ Datos y Base de Datos

- ◆ Repository (Repositorio): Habla directamente con la base de datos para buscar, guardar, actualizar o borrar datos.
- ◆ Entity (Entidad): Define la estructura de las tablas de la base de datos como objetos de código.

→ Configuración

- ◆ Config (Configuración): Guarda los datos fijos, las conexiones y los permisos iniciales para que el sistema funcione.

Mientras que el folder de resources contiene la vista o el frontend.

- Static: Aquí se colocan imágenes, hojas de estilo, fuentes, código javascript que se ejecuta en el cliente, entre otras cosas.
- Templates: Plantillas thymeleaf y código javascript que se resuelven y ejecutan del lado del servidor para producir html.

Spring annotations

Son metadatos basados en texto añadidos al código Java para configurar el comportamiento del framework Spring. Algunas son:

Core Boot & Configuration

- `@SpringBootApplication`: Marks the main application class. It automatically triggers `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan`.
- `@Configuration`: Defines a class as a source of bean definitions.
- `@Bean`: Placed on methods inside configuration classes to explicitly declare a Spring-managed bean.
- `@ComponentScan`: Instructs Spring to scan designated packages for registered beans.
- `@Value`: Injects application property values into variables.
- `@ConfigurationProperties`: Maps external properties (like YAML files) to a type-safe POJO class.

Dependency Injection & Stereotypes

- `@Component`: A generic annotation that marks any class as a Spring-managed component for auto-detection.
- `@Autowired`: Automatically injects dependencies by data type into constructors, fields, or setter methods.
- `@Qualifier`: Used alongside `@Autowired` to specify exactly which bean to inject when multiple matches exist.
- `@Primary`: Declares a default fallback bean when multiple candidate beans share the same type.
- `@Lazy`: Delays bean initialization until the exact moment it is requested by the system.

Spring annotations

Web & REST Endpoints (Spring MVC)

- `@RestController`: Simplifies REST API design by combining `@Controller` and `@ResponseBody`.
- `@RequestMapping`: Maps HTTP web requests to specific controller paths and handler methods.
- Shortcuts (`@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`): Handle method-specific HTTP routing requests directly.
- `@RequestBody`: Binds the incoming JSON HTTP request body to a structured Java object parameter.
- `@PathVariable`: Extracts dynamic values out of the URI path pattern.
- `@RequestParam`: Captures query parameters passed inside the request URL string.

Spring Boot Specifics

These manage auto-configurations and properties inside a Spring Boot framework context. [[1](#), [2](#)]

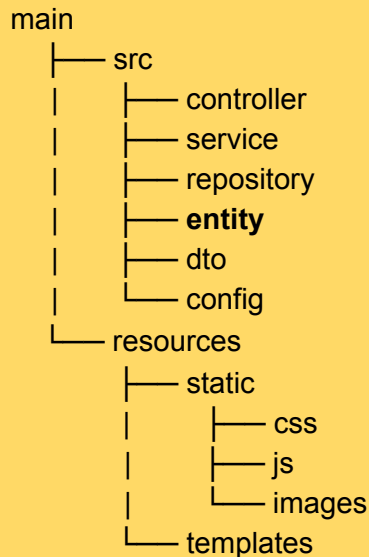
- `@SpringBootApplication`: Combines `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan`.
- `@ComponentScan`: Instructs Spring where to look for component stereotypes.
- `@ConfigurationProperties`: Binds structured properties configuration directly to type-safe objects.

Spring MVC

Módulo del framework Spring que sirve para crear páginas y servicios web con el patrón Model-View-Controller.

- Model. Representa los datos del dominio de la aplicación que viajan entre el servidor y el cliente.
- View. Responsable de presentar en la interfaz del usuario el modelo. En aplicaciones web son salidas HTML, en este curso por medio de plantillas Thymeleaf procesadas del lado del servidor. Spring también soporta otras tecnologías de plantillas como Apache FreeMaker o Mustache, entre otras.
- Controller. Procesa las solicitudes HTTP de los usuarios. Administra el flujo de la aplicación. Construye un modelo apropiado ya sea para pasarlo a la vista o para recibirlo de la vista y procesar los datos por medio de las reglas del negocio.

Las entities



Para nuestro primer microservicio solo necesitamos una clase Java que represente el modelo de datos de la entidad Category:

```
package mx.tecnm.cdmadero.product.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

import java.math.BigDecimal;

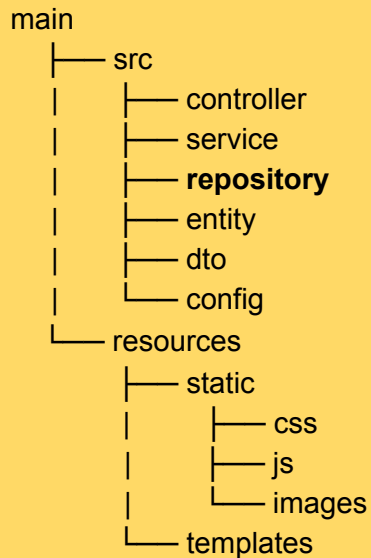
@Entity
public class Category {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    Integer categoryId;
    String categoryName;
    String description;
    BigDecimal tax;
    String picture;

    public Category() {
        this.categoryId = null;
    }

    //Otros constructores, getters y setters
}
```

Los repositories



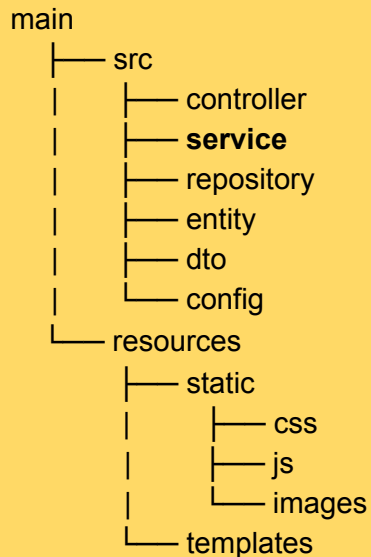
El repositorio, en la mayoría de los casos, es gestionado por completo por la JPA (Java Persistence API) con la simple declaración de una subinterfaz vacía que derive de CrudRepository:

```
package mx.tecnm.cdmadero.product.repository;

import mx.tecnm.cdmadero.product.entity.Category;
import org.springframework.data.repository.CrudRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface CategoryRepository extends
CrudRepository<Category, Integer> {
}
```

Los services



El servicio, se encarga de la lógica del negocio incluyendo en ella las validaciones:

```
package mx.tecnm.cdmadero.product.service;

import mx.tecnm.cdmadero.product.entity.Category;
import mx.tecnm.cdmadero.product.repository.CategoryRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.ArrayList;

@Service
public class CategoryService {
    @Autowired
    CategoryRepository categoryRepository;

    public ArrayList<Category> getAll() {
        return (ArrayList<Category>)
this.categoryRepository.findAll();
    }
}
```

Diseño de URLs

Una URL representa recursos, no acciones. Las acciones se determinan por la operación HTTP a realizar. Debido a que las APIs evolucionan se debe incluir en la URL la versión a la que corresponde.

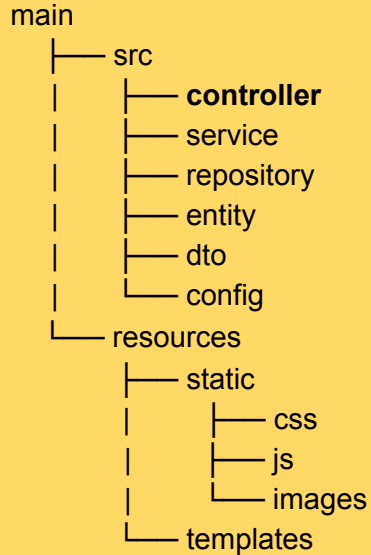
→ Ejemplos de URLs correctas:

- ◆ /api/v1/products
- ◆ /api/v1/categories
- ◆ /api/v1/products/7508006633435
- ◆ /api/v2/categories

→ Ejemplos de URLs incorrectas:

- ◆ /products
- ◆ /api/v1/getProducts

Los controllers



Los controladores son la interfaz entre el cliente y el servicio web. Una Rest API para intercomunicación entre aplicaciones solo requiere de RestControllers:

```
package mx.tecnm.cdmadero.product.controller;

import mx.tecnm.cdmadero.product.entity.Category;
import mx.tecnm.cdmadero.product.service.CategoryService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.ArrayList;

@RestController
@RequestMapping("/api/v1/categories")
public class CategoryController {

    @Autowired
    CategoryService categoryService;

    @GetMapping
    public ArrayList<Category> getCategories(){
        return this.categoryService.getAll();
    }
}
```

ResponseEntity

En lugar de devolver únicamente el objeto, es recomendable devolver una respuesta HTTP completa.

Incorrecto:

```
@GetMapping("/{id}")  
public ProfesorDTO obtener(@PathVariable Long id){  
    return service.buscar(id);  
}
```

Correcto:

```
@GetMapping("/{id}")  
public ResponseEntity<ProfesorDTO> obtener(  
    @PathVariable Long id){  
  
    return ResponseEntity.ok(  
        service.buscar(id)  
    );  
}
```

Configuración

Las aplicaciones Spring Boot deben configurarse para que puedan operar correctamente, definiendo valores como puerto de comunicaciones, conexión a base de datos, identificador de la aplicación, entre otros.

La configuración se puede llevar a cabo por diferentes medios en el orden de preferencia mostrado a continuación:

1. **Argumentos de la línea de comandos** (ej., `--server.port=9000`)
2. **Propiedades Java del sistema** (`System.getProperties()`)
3. **Variables de ambiente del sistema operativo** (ej., `SERVER_PORT=9000`)
4. **Archivo de configuración** (`application.properties` o `application.yaml` dentro del compilado jar)

Configuración en configuration.properties

1. Server Settings

server.port = 8081

server.servlet.context-path = /api

2. Application Identity

spring.application.name = user-service

3. Database Configurations (MySQL Example)

spring.datasource.url = jdbc:mysql://localhost:3306/mydb

spring.datasource.username = auser

spring.datasource.password = secret_pwd

spring.datasource.driver-class-name = com.mysql.cj.jdbc.Driver

4. JPA / Hibernate Configuration

spring.jpa.hibernate.ddl-auto = update

spring.jpa.show-sql = true

Configuración en configuration.yamal

```
spring:
  server:
    port: 8081
  servlet:
    context-path: /api
  application:
    name: user-service
  datasource:
    url: jdbc:sqlserver://localhost:1433;encrypt=false;databaseName=dbcategories
    username: auser
    password: secret_pwd
    driver-class-name: com.microsoft.sqlserver.jdbc.SQLServerDriver
```

Frontend con Spring Thymeleaf

Spring Thymeleaf es un motor de plantillas para aplicaciones Java/Kotlin del lado del servidor que forma parte del patrón Modelo-Vista-Controlador de Spring para generar páginas web dinámicas HTML.

Se requiere:

- Incluir la dependencia correspondiente al archivo de compilación Maven o Gradle
- Crear el controlador que manejará la petición HTTP, procesará los datos y los pasará a la plantilla
- Codificar la plantilla thymeleaf

¿Por qué utilizar Thymeleaf?

Porque:

- Está integrado con Spring Boot.
- Utiliza HTML estándar.
- Es sencillo de aprender

<https://www.thymeleaf.org/>

<https://www.thymeleaf.org/doc/tutorials/3.0/usingthymeleaf.html>

Spring MVC

Spring MVC es un módulo de Spring Framework que permite crear páginas web y servicios en Java. Usa el modelo Modelo-Vista-Controlador para separar los datos, el diseño visual y las acciones del usuario.

Componentes principales:

- Modelo: Guarda la información y los datos de la aplicación.
- Vista: Muestra la información en la pantalla, por ejemplo, con HTML.
- Controlador: Recibe lo que pide el usuario y conecta los datos con la vista.

Funcionamiento del flujo:

- El usuario hace una petición por internet (abrir una página).
- El sistema la recibe y busca qué parte del código debe atenderla.
- El controlador procesa los datos y elige qué vista mostrar.
- La página web se regresa al navegador del usuario.

La vista con plantillas Thymeleaf

```
<!DOCTYPE html>  
  
<html xmlns:th="http://www.thymeleaf.org">  
  
  <body>  
  
    <h1 th:text="${titulo}">Título principal de la página</h1>  
  
  </body>  
  
</html>
```

El controlador en Spring MVC

```
import org.springframework.stereotype.Controller  
import org.springframework.ui.Model
```

```
@Controller  
public class HomeController {  
    @GetMapping("/")  
    public String inicio(Model model){  
        model.addAttribute("titulo", "MVC Spring")  
        return "index";  
    }  
}
```

Diferencia entre @RestController y @Controller

@RestController	@Controller
Devuelve JSON	Devuelve HTML
APIs REST	Aplicaciones Web

Comunicación entre microservicios

Una arquitectura orientada a servicios soporta la conexión o el compartimiento de recursos y datos de un modo flexible y estandarizado. Los servicios se describen y organizan para dar soporte a su descubrimiento automatizado y dinámico y a su reutilización.

Sus componentes son:

- Protocolo HTTP: Funciona como el canal de transporte base para enviar y recibir las solicitudes entre los sistemas.
- Formatos de datos: Emplean JSON por su ligereza y compatibilidad moderna, o XML para estructuras de datos más estrictas y validadas.
- Arquitectura RESTful: Utiliza los métodos de HTTP (GET, POST, PUT, DELETE) para administrar recursos de manera sencilla y rápida.

Comunicación entre microservicios en Spring Boot

Las herramientas que se pueden utilizar para comunicar microservicios con Spring Boot son:

- **RestClient:** Cliente HTTP síncrono moderno y fluido.
- **WebClient:** Cliente web reactivo y no bloqueante ideal para arquitecturas de alta concurrencia y WebFlux.
- **Spring Cloud OpenFeign:** Cliente declarativo que permite llamar a otros servicios mediante interfaces de Java con anotaciones.
- **RestTemplate:** Cliente HTTP clásico y síncrono, en desuso pero presente en sistemas heredados.

Comunicación entre microservicios con RestTemplate

Fue el primer cliente HTTP incluido en Spring, requiere agregar un Bean:

```
@Configuration
public class AppConfig {
    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

Comunicación entre microservicios con RestTemplate

Y consumir el servicio:

```
package mx.cdmadero.tecnm.category.service;  
import ...
```

```
@Service  
public class CategoryService {  
    @Autowired  
    RestTemplate restTemplate;  
  
    public CategoryService() {  
    }  
  
    public Category getCategory(Integer categoryId){  
        return restTemplate.getForObject("http://localhost:8080/api/v1/category/" + categoryId, Category.class);  
    }  
}
```

Comunicación entre microservicios con WebClient

Se usa para comunicación asíncrona:

```
@Bean  
public WebClient webClient(){  
    return WebClient.builder().build();  
}
```

Comunicación entre microservicios con WebClient

Se usa para comunicación asíncrona. Su bean de configuración es:

```
@Bean
public WebClient webClient(){
    return WebClient.builder().build();
}
```

Comunicación entre microservicios con WebClient

Y este es un ejemplo de su uso:

```
@Service
public class CursoClient {
    private final WebClient webClient;

    public CursoClient(WebClient webClient){
        this.webClient = webClient;
    }

    public List<CursoDTO> obtenerCursos(Long profesorId){
        return webClient.get().uri("http://localhost:8082/api/cursos/profesor/{id}", profesorId).retrieve().bodyToFlux(CursoDTO.class)
            .collectList()
            .block();
    }
}
```

Comunicación entre microservicios con OpenFeign

No configuramos, si no que definimos una interfaz:

```
import org.springframework.cloud.openfeign.FeignClient;
```

```
@FeignClient(  
    name="categories-service",  
    url="http://localhost:8082"  
)
```

```
public interface CategoriesClient {  
    @GetMapping("/api/v1/categories/{id}")  
    CategoryDTO getCategory(  
        @PathVariable Integer id  
    );  
}
```


Comunicación entre microservicios con OpenFeign

Y este es un ejemplo de su uso:

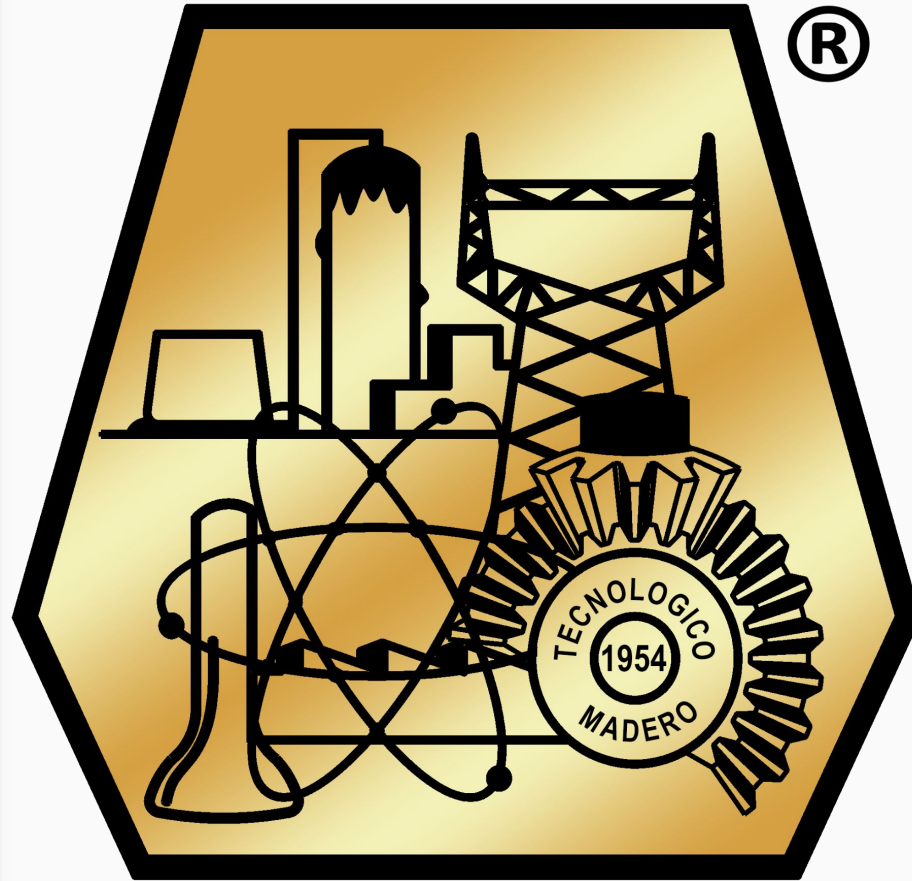
```
@Service
public class CategoryService{
    private final CategoryClient categoryClient;

    public CategoryService(categoryClient cursoClient){
        this.categoryClient = categoryClient;
    }
}
```

"A probar la comunicación de los web
services"



¡Gracias!



“POR MI PATRIA Y POR MI BIEN”